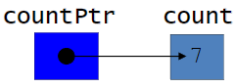
Variables tipo Puntero

- Las variables "normales" contienen una valor específico (*referencia directa*).
- Un *puntero* contiene la *dirección de memoria* de una variable "normal" (*referencia indirecta*)



Usos de los Punteros

- Simulan llamadas por referencia
- Permiten el manejo dinámico de la memoria
- Permiten la implementación de estructuras de datos dinámicas
- Permiten implementar arrays

Definición e Inicialización de variables Puntero

- ***Definición de Punteros:***
  - $tipoDatoPuntero\ *\ nombrePuntero1,\ *\ nombrePuntero2;$
  - Se pueden definir punteros a cualquier tipo de dato.
- ***Inicialización de Punteros:***
  - Los punteros se inicializan a 0 (*NULL*), o a una dirección de memoria concreta.
  - 0 o NULL indican que el puntero no apunta a nada.

Operadores de Punteros

- ***& (operador de dirección):***
  - Retorna la dirección del operando

$$int\ y\ =\ 5;$$
$$int\ *\ yPtr;$$
$$yPtr\ =\ \&y;$$

- *** (operador de indirección | desreferencia):**
  - Retorna el contenido de la variable cuya *dirección está almacenada* en el puntero.

Los operadores & y * son *complementarios*. Es decir:

$$cout\ <<\ \&\ *\ Ptr;\ |\ cout\ <<\ *\ \&Ptr;$$

Ambos devuelven lo mismo: la dirección de memoria almacenada en *Ptr*.

Expresiones con punteros y Aritmética de punteros

- *Operaciones aritméticas que se pueden realizar sobre punteros*:
  - *Incrementar | Decrementar* un puntero (++ , --)
  - *Sumar | Restar* un entero a un puntero (+ , += , - , -=)

Estas operaciones implica que a la dirección de memoria almacenada en el puntero se le sumen o resten tantos *bytes como corresponda al tipo de datos apuntado*.

ej. Supongamos que tengo un puntero a *int* (*4 bytes*) llamado *vPtr* y apunta a la primera dirección de memoria de un arreglo de 5 enteros, cuya dirección es 3000.

$$vPtr\ +=\ 2;\ //\ Esta\ operación\ es\ equivalente\ a\ vPtr\ =\ 3000\ +\ (4\ *\ 2);$$

donde 4 es el *tamaño en bytes* de las variables tipo *int* y 2 es la cantidad sumada.



- Los punteros se pueden *restar entre ellos*.
  - Esta resta devuelve el *número de elementos* entre uno y otro.
    ej. Tengo dos punteros que apuntan a un mismo arreglo, vPtr = &v[0]; y vPtr2 = &v[2]; La operación:

    $$vPtr2 - vPtr$$

    retorna 2.
- En general estas operaciones no tienen sentido a menos que estemos trabajando sobre *arreglos*.
- Los punteros del mismo tipo se pueden *asignar entre ellos*.
- Si no son del mismo tipo, se debe usar un operador *cast*.
- Para estos casos hay excepciones y son los *punteros genéricos*.

Punteros Genéricos | $void * Ptr$

- No se asignan a *ningún tipo de dato*, sino que apuntan a cualquier tipo de dato.
- No se precisa un *casting* para convertir cualquier puntero a un puntero void.
- *No pueden ser desreferenciados*.

Relación entre Punteros y Arreglos

- Un *nombre de arreglo* es un *puntero constante*, pues contiene la dirección de memoria del primer elemento del arreglo y no se puede modificar.
- Los punteros se pueden utilizar para acceder a un arreglo mediante subíndices.
- Dado un arreglo b[5]; y un puntero *bPtr*:
  - $bPtr = b$; // Asigna la dirección de memoria del primer elemento del arreglo en el puntero *bPtr*.
    - El elemento b[3]; puede ser accedido mediante:
      - *(bPtr + 3) | Notación puntero / desplazamiento
      - bPtr[3] | Notación puntero / subíndice
      - *(b + 3) | Aritmética de punteros sobre el arreglo
  - $bptr = \&b[3]$; // Asigna la dirección de memoria del elemento con índice 3 sobre el arreglo en el puntero *bPtr*.